

DATA-260— Homework 3

Xu Wang

Repository: github.com/xuwang-goldenwater/data260-9275

Section 0 — Personal configuration and domain

Value	This submission	How it is derived
SID4	9275	last four digits of the SJSU Student ID
PORT_BASE	8275	$8000 + (9275 \bmod 900)$
PREFIX	s9275	"s" + SID4
SEED	9275	SID4
VERIFY_SEED	269275	$260000 + \text{SID4}$
DOMAIN_ID	3 — grocery supply and recall notices	$9275 \bmod 8$

Item	Value
Hardware	MacBook Pro, Apple silicon, 8 GB RAM, macOS
Local model	sentence-transformers/all-MiniLM-L6-v2 (embeddings, 384 dimensions, CPU). No LLM is used in HW3.
Python	3.12, conda environment "data260"
Tagged commit	6712ca0 (code, data and evidence)
Tag	hw3

Homework 3 extends the same codebase. Homework 1 and 2 files are unchanged; the new code is in `code/auth_app/` (Part 1) and `code/rag_*.py` (Part 2). `reports/hw03/` holds evidence only: corpus snapshot, questions, raw output, metrics, logs and verification.

Part 1 — FastAPI authentication app

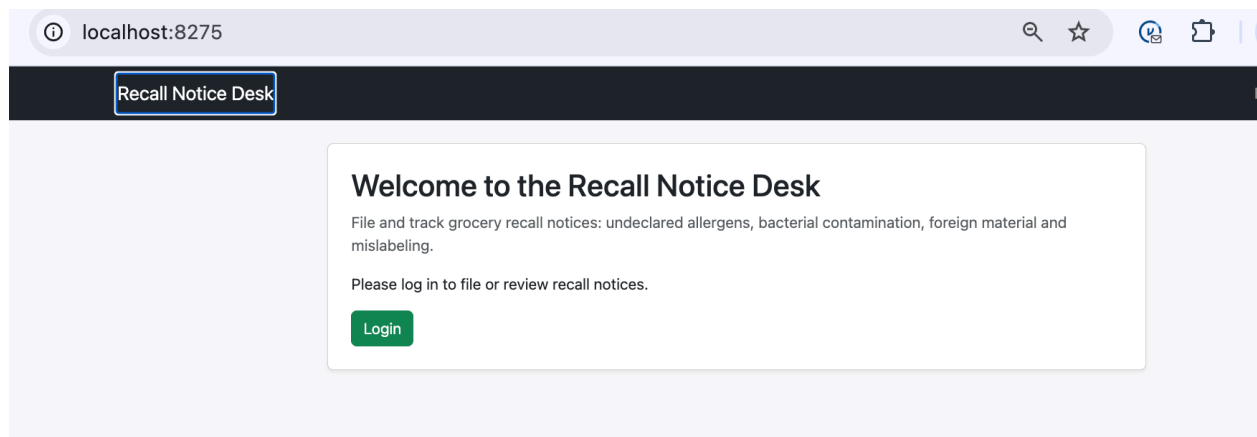
1.1 What was changed from the starter code

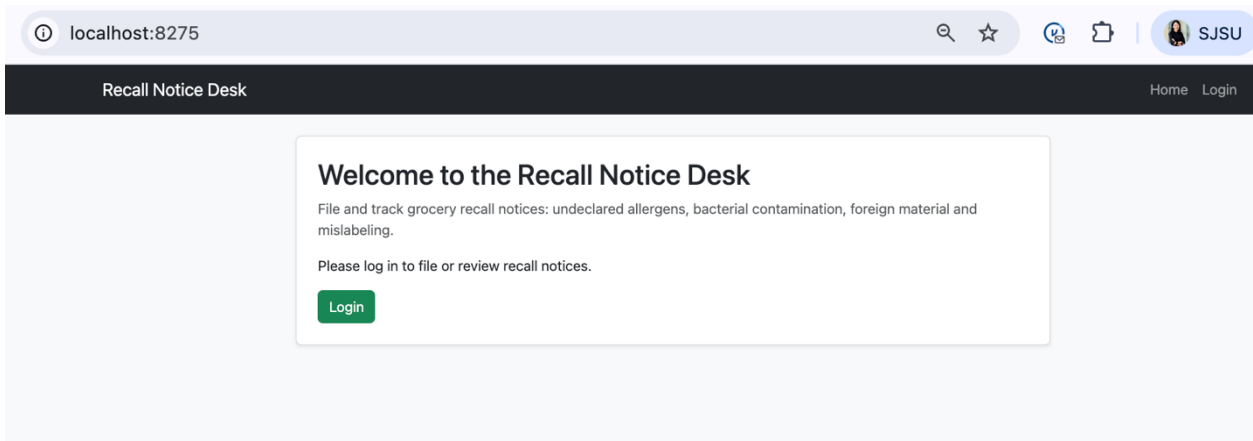
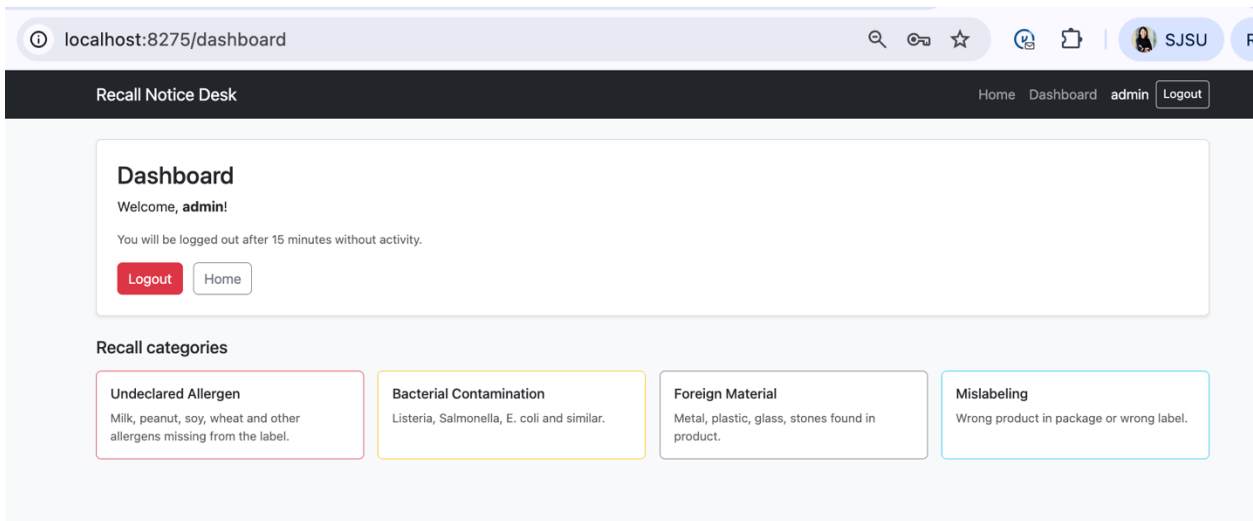
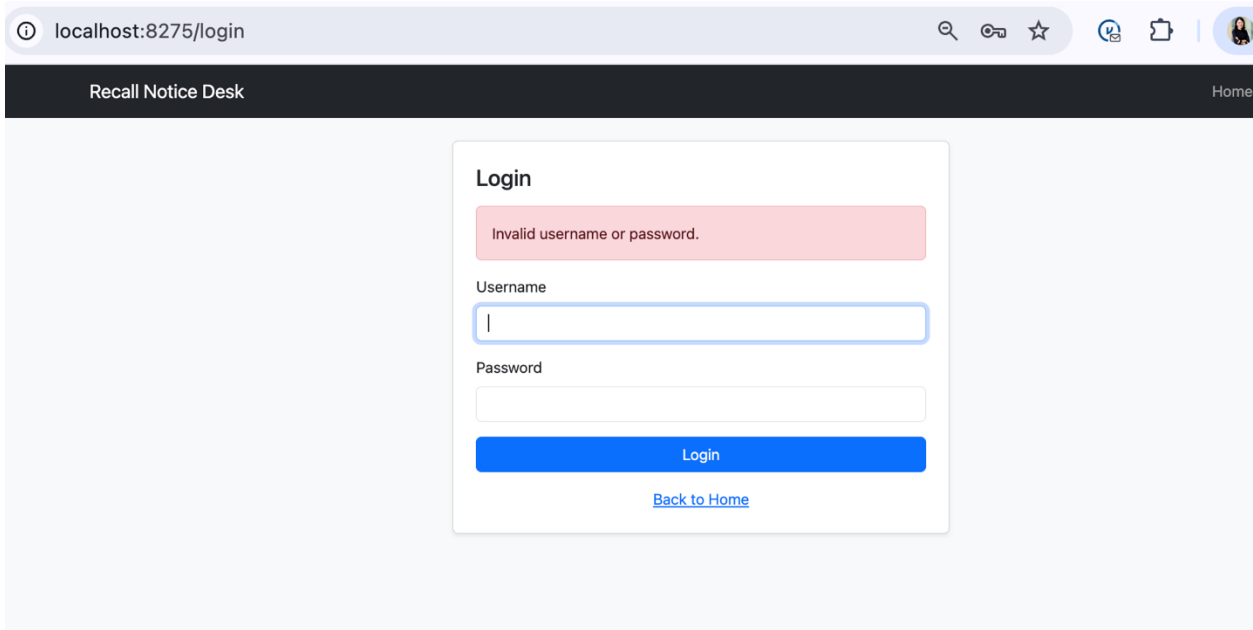
The app starts from the instructor's HW3 starter (`main.py`, `routers/auth.py`, three templates). It is called **Recall Notice Desk**, the login-protected front door of the recall-notice application from Homework 1 and 2.

- Runs on `PORT_BASE` **8275** instead of 8000, and can be started from the repo root (`make run-auth`).
- Templates are loaded by absolute path, so the app works from any working directory.
- Failed login re-renders the login page with HTTP 401 and a Bootstrap **alert-danger** message.
- The password is checked with **bcrypt**; only the hash is kept in memory.
- The cookie carries only a random session id. A server-side table (`ACTIVE_SESSIONS`) decides whether that id is still valid — this is what makes logout and the idle timeout real (section 1.3).
- All pages extend one `base.html` with a responsive Bootstrap navbar.

1.2 Routes — one screenshot per route

All routes are defined in one `APIRouter` in `code/auth_app/routers/auth.py` (full listing in 1.5). Log in with **admin / password**. Use Chrome at <http://localhost:8275>.





1.3 Session management

Secure cookie. SessionMiddleware is configured with `https_only=True` (adds Secure) and `same_site="lax"` (adds SameSite=lax); Starlette always adds `HttpOnly`. So the cookie is never sent over plain HTTP to a non-local host, cannot be read by JavaScript, and is not sent on cross-site POSTs.

```
((base) Sunny@MacBook-Pro-7 ~ % curl -si -X POST -d "username=admin&password=password" http://127.0.0.1:8275/login | grep -i set-cookie
set-cookie: session=eyJzaWQiOiAiRWE2Nzg4U29ydEdlZjZ4dFZwS19YTC1ibmpzckxtT0ZM0GVkMVo2XzZjTSJ9.arFb8w.rFuurp87TQKvVHzRIQYf4LwXsYU; path=/; Max-Age=3600; httponly; samesite=lax; secure
```

Why logout alone was not enough. Starlette stores the session *inside* the signed cookie. The starter's logout only told the browser to drop the cookie; a copy of the old cookie would still pass the signature check and open /dashboard until `max_age` ran out. The fix is a server-side table: the cookie holds only a random `sid`, and /dashboard accepts it only if the `sid` is still in `ACTIVE_SESSIONS`. Logout deletes the `sid`, so the old cookie becomes useless.

Idle timeout. Every request refreshes `last_seen` for the `sid`. If more than `IDLE_TIMEOUT_SECONDS` (15 minutes) pass without a request, the next request deletes the `sid` and redirects to `/login?reason=ended`. `max_age=3600` stays as a hard one-hour upper limit even for an active user.

Proof. `make demo-session` runs the real app in-process and (1) prints the Set-Cookie header, (2) opens /dashboard, (3) logs out and replays the saved old cookie, (4) waits past a 3-second idle limit.

```
((base) Sunny@MacBook-Pro-7 ~ % cd ~/Documents/data260-9275
conda activate data260
make demo-session
[python code/auth_session_demo.py
/opt/miniconda3/envs/data260/lib/python3.12/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient # noqa
== 1. Login ==
POST /login (correct password) -> 302 /dashboard
Set-Cookie: session=eyJzaWQiOiAiRWE2Nzg4U29ydEdlZjZ4dFZwS19YTC1ibmpzckxtT0ZM0GVkMVo2XzZjTSJ9.arFb8w.rFuurp87TQKvVHzRIQYf4LwXsYU; path=/; Max-Age=3600; httponly; samesite=lax; secure

== 2. Use the session ==
GET /dashboard with the cookie -> 200

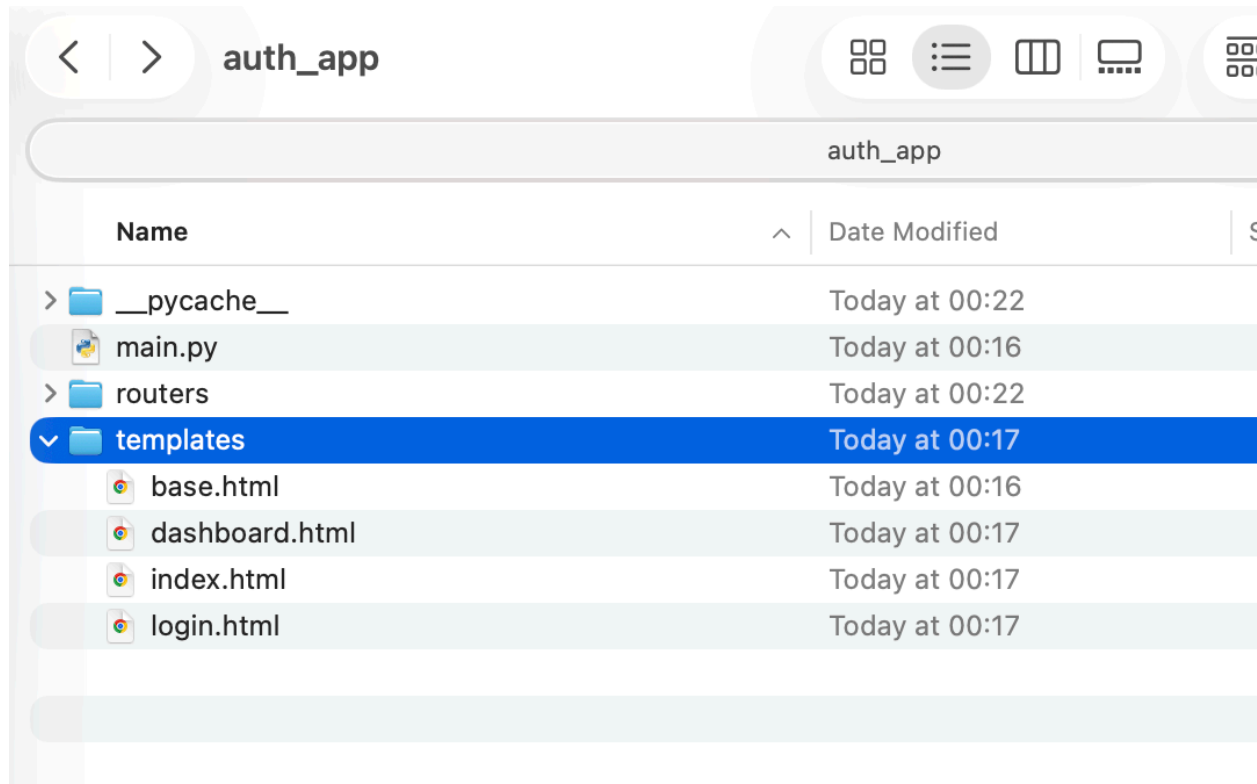
== 3. Logout, then replay the old cookie ==
GET /logout -> 302 /
GET /dashboard with the saved OLD cookie -> 302 /login?reason=ended

== 4. Idle timeout ==
GET /dashboard right after login -> 200
... waiting 4 seconds without any request ...
GET /dashboard after 4 idle seconds -> 302 /login?reason=ended
(data260) Sunny@MacBook-Pro-7 data260-9275 %
```

1.4 Styling with Bootstrap

- **Navbar** — dark, collapses into a toggler on narrow screens; shows Login or Dashboard/username/Logout depending on the session.
- **Cards** — the welcome box, the login form (centred, col-md-6) and the dashboard; four recall-category cards in a responsive grid (col-sm-6 col-lg-3).
- **Buttons** — primary / success / danger / outline variants for the actions.
- **Alerts** — alert-danger for a failed login, alert-warning when a session has ended.

1.5 Templates directory and auth.py



code/auth_app/routers/auth.py

```
"""
Routes for HW3 Part 1: /, /login, /dashboard, /logout.

Why there is a server-side session table (ACTIVE_SESSIONS):
Starlette's SessionMiddleware keeps the session *inside* the signed cookie.
If we only did request.session.clear() on logout, someone who saved the old
cookie could send it again and still reach /dashboard until max_age runs out.
So the cookie only carries a random session id ("sid"), and the server keeps
the list of sids that are still valid. Logout and idle timeout delete the sid
from that list, which makes the old cookie useless.
"""
import os
import secrets
import time
from pathlib import Path

import bcrypt
from fastapi import APIRouter, Form, Request
from fastapi.responses import RedirectResponse
```

```

from fastapi.templating import Jinja2Templates
from starlette.status import HTTP_302_FOUND

router = APIRouter()

# Absolute path, so the app works no matter which folder it is started from
TEMPLATE_DIR = Path(__file__).resolve().parent.parent / "templates"
templates = Jinja2Templates(directory=str(TEMPLATE_DIR))

# Log out a user who has not made a request for this many seconds
IDLE_TIMEOUT_SECONDS = int(os.getenv("IDLE_TIMEOUT_SECONDS", "900")) # 15 minutes

# Demo user. Only the bcrypt hash of the password is kept in memory.
# A real application would load users from a database.
USERS = {
    "admin": bcrypt.hashpw(b"password", bcrypt.gensalt()),
}

# sid -> {"user": username, "last_seen": unix time}
ACTIVE_SESSIONS = {}

def get_current_user(request: Request):
    """Return the logged-in username, or None.

    Also enforces the idle timeout and refreshes last_seen on every request.
    """
    sid = request.session.get("sid")
    if not sid:
        return None

    record = ACTIVE_SESSIONS.get(sid)
    if record is None:
        # sid was logged out (or the server restarted) -> cookie is dead
        request.session.clear()
        return None

    idle_for = time.time() - record["last_seen"]
    if idle_for > IDLE_TIMEOUT_SECONDS:
        # idle too long -> end the session on the server side
        del ACTIVE_SESSIONS[sid]
        request.session.clear()
        return None

    record["last_seen"] = time.time()
    return record["user"]

def check_password(username: str, password: str) -> bool:
    hashed = USERS.get(username)
    if hashed is None:
        return False
    return bcrypt.checkpw(password.encode("utf-8"), hashed)

@router.get("/")
def home(request: Request):
    user = get_current_user(request)
    return templates.TemplateResponse(
        request, "index.html", {"user": user}
    )

@router.get("/login")
def login_page(request: Request, reason: str = ""):

```

```

user = get_current_user(request)

info = None
if reason == "ended":
    info = "Your session has ended (idle timeout or logout). Please log in again."

return templates.TemplateResponse(
    request, "login.html", {"user": user, "error": None, "info": info}
)

@router.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    if check_password(username, password):
        # Start a fresh session with a new random id
        request.session.clear()
        sid = secrets.token_urlsafe(32)
        ACTIVE_SESSIONS[sid] = {"user": username, "last_seen": time.time()}
        request.session["sid"] = sid
        return RedirectResponse(url="/dashboard", status_code=HTTP_302_FOUND)

    # Wrong username or password: show the login page again with an alert
    return templates.TemplateResponse(
        request,
        "login.html",
        {"user": None, "error": "Invalid username or password.", "info": None},
        status_code=401,
    )

@router.get("/dashboard")
def dashboard(request: Request):
    had_sid = "sid" in request.session
    user = get_current_user(request)

    if not user:
        url = "/login?reason=ended" if had_sid else "/login"
        return RedirectResponse(url=url, status_code=HTTP_302_FOUND)

    return templates.TemplateResponse(
        request,
        "dashboard.html",
        {"user": user, "idle_minutes": IDLE_TIMEOUT_SECONDS // 60},
    )

@router.get("/logout")
def logout(request: Request):
    sid = request.session.get("sid")
    if sid:
        ACTIVE_SESSIONS.pop(sid, None) # old cookie can no longer be used
    request.session.clear()
    return RedirectResponse(url="/", status_code=HTTP_302_FOUND)

```

Part 2 — Three LlamaIndex chunking techniques (retrieval only)

2.1 Corpus

The graded dataset is a local snapshot of public documents for DOMAIN_ID 3:

- **89 FDA company recall announcements** for food and beverages (Nov 2025 – Sep 2026), from [fda.gov/safety/recalls-market-withdrawals-safety-alerts](https://www.fda.gov/safety/recalls-market-withdrawals-safety-alerts). They cover the four categories of the domain schema — undeclared allergen, bacterial contamination, foreign material, mislabeling — plus toxins, lead, pesticide and radioactive contamination. Pet food and drugs sold as supplements were left out.
- **21 CFR Part 7** (Enforcement Policy) from the eCFR, version of 2026-09-17. Subpart C is FDA’s recall regulation: recall classes, strategy, communications, status reports, termination.

`code/rag_fetch_corpus.py` downloads each URL in `reports/hw03/urls.txt` once and keeps only the title, the Summary box and the announcement text (no menus, footers or photo captions), one `.txt` file per page. It then writes `CORPUS_MANIFEST.json` (file, bytes, SHA-256, source URL) and `SOURCES.md` (URL and access date).

The mix is deliberate. The announcements are short and follow the same template (“X is voluntarily recalling Y because...”), which is realistic and makes confident-but-wrong retrievals likely. Part 7 is one long, structured legal text where chunk boundaries matter.

	Value
Files	90 (89 FDA announcements + 21 CFR Part 7)
Total size	274,233 bytes
Largest file	<code>ecfr_21cfr_part7.txt</code> , 37,649 bytes

```
[80/90] 1924 bytes fda_carrot-top-kitchens-bridgeport-formerly-redding.txt
[81/90] 2723 bytes fda_direct-source-seafood-llc-recalling-frozen.txt
[82/90] 2533 bytes fda_tbc-distribution-corporate-recalls-lucky-foods.txt
[83/90] 2450 bytes fda_prairie-farms-announces-recall-prairie-farms.txt
[84/90] 2358 bytes fda_wegmans-recalls-locatelli-grated-pecorino-romano.txt
[85/90] 2228 bytes fda_mamtakim-inc-recalls-dried-whole-smelts.txt
[86/90] 2556 bytes fda_james-skinner-llc-issues-voluntary-recall.txt
[87/90] 1588 bytes fda_lunds-byerlys-voluntary-recalls-lb-monster.txt
[88/90] 2621 bytes fda_white-castle-frozen-food-division-recalls.txt
[89/90] 2785 bytes fda_mellace-family-brands-california-inc-recalls.txt
[90/90] 37649 bytes ecfr_21cfr_part7.txt
```

```
90 files, 274,233 bytes -> CORPUS_MANIFEST.json, SOURCES.md
```

2.2 Questions (committed before the retrieval run)

`reports/hw03/questions.yaml` holds five questions, each with an expected answer and expected source file. All five depend on one source document only. The file was committed in `cb591c6` before the retrieval results were committed in `6712ca0`; `verify_hw03.py` checks this order from git history.

ID	Question	Expected source
q1	What does a Class II recall mean in FDA's recall classification?	ecfr_21cfr_part7.txt
q2	How often should a recalling firm send recall status reports to FDA?	ecfr_21cfr_part7.txt
q3	Why does preparing the recalled a2 Platinum infant formula with hot water not make it safe?	fda_a2-platinum-usa-label-infant-formula.txt
q4	Which recalled seafood may be contaminated with cesium-137, and where was it imported from?	fda_direct-source-seafood-llc-recalling-frozen.txt
q5	Which milk was recalled because it may contain food-grade cleaning agents?	fda_prairie-farms-announces-recall-prairie-farms.txt

2.3 Setup and the three chunkers

Installed: llama-index-core, llama-index-embeddings-huggingface, sentence-transformers, faiss-cpu, numpy, pandas. The embedding model is the public sentence-transformers/all-MiniLM-L6-v2, set as Settings.embed_model so LlamaIndex never falls back to its OpenAI default. Each file becomes one Document; the file name is kept as metadata but excluded from the embedding, so a chunk's vector depends on its text only.

Technique	LlamaIndex parser	Parameters and why
Token	TokenTextSplitter	chunk_size=200, chunk_overlap=30. MiniLM reads at most 256 word pieces, so 200 tokens keeps a chunk inside what the model actually sees; 15% overlap keeps sentences cut at a boundary retrievable.
Semantic	SemanticSplitterNodeParser	buffer_size=1, breakpoint_percentile_threshold=95, using MiniLM. Splits where the embedding distance between neighbouring sentence groups is in the top 5%.
Sentence window	SentenceWindowNodeParser	window_size=3. Each sentence is one node; the 3 sentences before and after are stored in metadata["window"].

code/rag_chunking.py — step 1, the only step that differs between techniques

```
def chunk_documents(technique, documents, embed_model):
    """Step 1 - the only step that differs between the three techniques."""
    if technique == "token":
        params = {"chunk_size": TOKEN_CHUNK_SIZE, "chunk_overlap": TOKEN_CHUNK_OVERLAP}
        parser = TokenTextSplitter(**params)
    elif technique == "semantic":
        params = {"buffer_size": SEMANTIC_BUFFER_SIZE,
                  "breakpoint_percentile_threshold": SEMANTIC_BREAKPOINT_PERCENTILE}
        parser = SemanticSplitterNodeParser(embed_model=embed_model, **params)
    elif technique == "sentence_window":
        params = {"window_size": WINDOW_SIZE}
        parser = SentenceWindowNodeParser.from_defaults(
            window_size=WINDOW_SIZE,
            window_metadata_key="window",
            original_text_metadata_key="original_text",
        )
    else:
        raise ValueError(technique)
    nodes = parser.get_nodes_from_documents(documents)
    return nodes, params
```

Each technique's nodes go into VectorStoreIndex(nodes, embed_model=embed_model), which uses the in-memory SimpleVectorStore by default.

2.4 Retrieval-only function

For a question and $k = 5$, `retrieve()` embeds the query and prints its dimension and first 8 values; runs the similarity search with the pre-computed query vector, so the timer measures the search only; embeds the returned chunks again explicitly and computes cosine similarity against the query; prints both vector shapes and the table rank / store_score / cosine_sim / chunk_len / preview; and returns one row per hit for `raw/retrievals.jsonl`.

code/rag_chunking.py — `retrieve()`

```
def retrieve(index, embed_model, technique, question_id, question, k=TOP_K):
    """Step 3 - retrieval-only helper. Prints everything and returns one row per hit."""
    # Query embedding
    query_vec = np.array(embed_model.get_query_embedding(question))
    print(f"\n[{technique}] {question_id}: {question}")
    print(f"query embedding dim = {query_vec.shape[0]}, first 8 = {np.round(query_vec[:8], 4).tolist()}")

    # Similarity search only. The query is already embedded, so the timer
    # measures the search, not the embedding model.
    retriever = index.as_retriever(similarity_top_k=k)
    start = time.perf_counter()
    hits = retriever.retrieve(QueryBundle(query_str=question, embedding=query_vec.tolist()))
    latency_ms = (time.perf_counter() - start) * 1000

    # Embed the returned chunks again, explicitly, and compare with the query
    texts = [chunk_text(hit.node) for hit in hits]
    doc_vecs = np.array(embed_model.get_text_embedding_batch(texts))
    print(f"query vector shape = {query_vec.shape}, stacked chunk vectors shape = {doc_vecs.shape}")
    print(f"similarity search took {latency_ms:.2f} ms")

    run_at = datetime.now().isoformat(timespec="seconds")
    rows = []
    for rank, (hit, text, vec) in enumerate(zip(hits, texts, doc_vecs), start=1):
        rows.append({
            "technique": technique,
            "question_id": question_id,
            "k": k,
            "rank": rank,
            "store_score": round(float(hit.score), 6),
            "cosine_sim": round(cosine(query_vec, vec), 6),
            "node_id": hit.node.node_id,
            "chunk_hash": hashlib.sha256(text.encode("utf-8")).hexdigest()[:16],
            "source_file": hit.node.metadata["source_file"],
            "chunk_len": len(text),
            "preview": " ".join(text.split())[:160],
            "latency_ms": round(latency_ms, 3),
            "run_at": run_at,
        })

    # The table the assignment asks for
    print(f"{'rank':>4} {'store_score':>11} {'cosine_sim':>10} {'chunk_len':>9} {'source_file':<45} "
          "preview")
    for r in rows:
        print(f"{'rank':>4} {r['store_score']:>11.4f} {r['cosine_sim']:>10.4f} {r['chunk_len']:>9} "
              f"{'source_file':<45} {r['preview']}")
    return rows, {"question_id": question_id, "dim": int(query_vec.shape[0]),
                  "first_8": np.round(query_vec[:8], 6).tolist()}
```

Sentence window: `cosine_sim` and `chunk_len` are measured on the single sentence that was indexed, not on the window, so that for all three techniques the recomputed cosine is comparable with the store score.

```
[token] q5: Which milk was recalled because it may contain food-grade cleaning agents?
query embedding dim = 384, first 8 = [-0.0391, -0.0944, 0.0283, 0.0426, 0.079, 0.0413, -0.0559, 0.0686]
query vector shape = (384,), stacked chunk vectors shape = (5, 384)
similarity search took 8.56 ms
rank store_score cosine_sim chunk_len source_file preview
1 0.6657 0.6657 608 fda_sks-copack-recalls-various-specialty-beve | 27-Oct-2027 BT371 | Boba Time | PISTACHIO POWDER 5/3.5# | 20260219 | 20-Aug-2027 FN1000 |
FANALE | UBE-TARO 10/2# | 20260223 | 24-Aug-2027 55500 | DENDA | MIL stores in Arizona, California, Connecticut, Massachusetts, New Jersey, New York, and Rhode
2 0.6545 0.6545 913 fda_whole-foods-market-issues-allergy-alert.t Inc. due to concerns regarding potential Salmonella contamination in milk powder. Out of an
Island. Recalled product was cut and plastic wrapped by the pound wit
3 0.6475 0.6475 772 fda_sugar-foods-issues-recall-specific-lots.t Inc. due to concerns regarding potential Salmonella contamination in milk powder. Out of an
abundance of caution, and because this milk powder was used in a sea affected lots of Lactolac Lactation Granola. The specific source of the contamination has not
4 0.6439 0.6439 933 fda_hampton-grocers-inc-recalls-its-8.txt
yet been determined. To date, neither the finished Lactolac product
5 0.6398 0.6398 908 fda_prairie-farms-announces-recall-prairie-fa Prairie Farms Announces Recall of Prairie Farms Gallon Fat Free Milk Company Announcement
Date: November 25, 2025 FDA Publish Date: November 26, 2025 Product Ty

[semantic] q5: Which milk was recalled because it may contain food-grade cleaning agents?
query embedding dim = 384, first 8 = [-0.0391, -0.0944, 0.0283, 0.0426, 0.079, 0.0413, -0.0559, 0.0686]
query vector shape = (384,), stacked chunk vectors shape = (5, 384)
similarity search took 5.73 ms
rank store_score cosine_sim chunk_len source_file preview
1 0.6726 0.6726 668 fda_prairie-farms-announces-recall-prairie-fa Prairie Farms Announces Recall of Prairie Farms Gallon Fat Free Milk Company Announcement
Date: November 25, 2025 FDA Publish Date: November 26, 2025 Product Ty CST, Monday-Friday). If consumers have an allergy or sensitivity to milk, they should not
2 0.6439 0.6439 410 fda_frito-lay-issues-voluntary-allergy-alert. consume the product and discard it immediately. Frito-Lay has informed No illnesses have been reported to date. The recall was initiated after it was discovered
3 0.6412 0.6412 766 fda_prince-bakery-inc-issues-allergy-alert.tx during routine FDA inspection that the Spanish Style bread, which incl for a certain lot of low heat nonfat dry milk powder due to possible Salmonella
4 0.6265 0.6265 811 fda_sks-copack-recalls-various-specialty-beve contamination. According to California Dairies, the recall followed the identifi This recall was initiated after foreign language packaging was discovered that did not
5 0.6213 0.6213 997 fda_lidl-us-recalls-eridanous-shortbread-cook contain English ingredients, nutrition facts, or allergen declarations. C

[sentence_window] q5: Which milk was recalled because it may contain food-grade cleaning agents?
query embedding dim = 384, first 8 = [-0.0391, -0.0944, 0.0283, 0.0426, 0.079, 0.0413, -0.0559, 0.0686]
query vector shape = (384,), stacked chunk vectors shape = (5, 384)
similarity search took 30.16 ms
rank store_score cosine_sim chunk_len source_file preview
1 0.7301 0.7301 154 fda_frito-lay-issues-voluntary-allergy-alert. Those with an allergy or severe sensitivity to milk run the risk of a serious or life-
threatening allergic reaction if they consume the recalled product. Out of an abundance of caution, and because this milk powder was used in a seasoning
2 0.7239 0.7239 160 fda_sugar-foods-issues-recall-specific-lots.t ingredient supplied to Sugar Foods, the company is initiating this recall. milk powder recall due to a concern of potential Salmonella contamination.
3 0.7172 0.7172 76 fda_champion-foods-recalls-some-batches-motor milk powder recall due to a concern of potential Salmonella contamination, which was supplied
4 0.7106 0.7106 179 fda_ghirardelli-chocolate-company-recalls-pow to a third-party manufacturer and used as an ingredient in powdere
5 0.7012 0.7012 528 fda_whole-foods-market-issues-allergy-alert.t Recalled product was cut and plastic wrapped by the pound with Whole Foods Market scale
labels and would be identified as follows: Type of Label | Product Descr
```

2.5 Machine-readable output

- `raw/retrievals.jsonl` — one line per technique × question × rank ($3 \times 5 \times 5 = 75$ lines): technique, question_id, k, rank, store_score, cosine_sim, node_id, chunk_hash, source_file, chunk_len, preview, latency_ms, run_at.
- `raw/chunk_stats.json` — per technique: number of chunks, average / min / max chunk length (characters), parameters, embedding model and dimension.
- `raw/query_embeddings.jsonl` — dimension and first 8 values of each query vector.
- `annotations.csv` — my manual label for whether a retrieved chunk contains the answer. It is keyed by (technique, question_id, source_file, chunk_hash): node_id is random on every run, while the hash of the chunk text is stable, so labels survive a re-run.

`code/rag_summary.py` reads only these files plus `questions.yaml` and recomputes the tables below into `METRICS.md` (make `rag-summary`).

`code/rag_summary.py` — metric definitions

```
def summary_table(rows, stats):
    per_question = rows.groupby(["technique", "question_id"]).agg(
        top1_cosine=("cosine_sim", "max"),
        mean_k_cosine=("cosine_sim", "mean"),
        hit=("hit", "any"),
        latency_ms=("latency_ms", "first"),
    ).reset_index()

    per_technique = per_question.groupby("technique").agg(
        top1_cosine=("top1_cosine", "mean"),
        mean_k_cosine=("mean_k_cosine", "mean"),
        recall_k=("hit", "mean"),
        latency_ms=("latency_ms", "mean"),
    ).reset_index()

    table = stats[["technique", "n_chunks", "avg_chunk_len"]].merge(per_technique, on="technique")
    return table, per_question
```

k = 5, averaged over the five questions. Top-1 cosine = highest cosine in the top-k; Recall@k = share of questions with at least one top-k chunk from the expected source file.

Technique	Chunks	Avg chunk length (chars)	Top-1 cosine	Mean@k cosine	Recall@k	Mean retrieval latency (ms)
Token	413	760.3	0.7085	0.6488	1.00	9.43
Semantic	204	1339.3	0.6867	0.6035	1.00	4.60
Sentence window	1706	160.2	0.7467	0.6817	0.80	29.85

```
data260) Sunny@MacBook-Pro-7 data260-9275 % make rag-summary
python code/rag_summary.py
### Retrieval quality (k = 5, averaged over 5 questions)

| Technique | Chunks | Avg chunk length (chars) | Top-1 cosine | Mean@5 cosine | Recall@5 | Mean retrieval latency (ms) |
|---|---|---|---|---|---|---|
| Token | 413 | 760.3 | 0.7085 | 0.6488 | 1.00 | 9.43 |
| Semantic | 204 | 1339.3 | 0.6867 | 0.6035 | 1.00 | 4.60 |
| Sentence window | 1706 | 168.2 | 0.7467 | 0.6817 | 0.80 | 29.85 |

### Per question: top-1 cosine / expected source found in top-k

| Technique | q1 | q2 | q3 | q4 | q5 |
|---|---|---|---|---|---|
| Token | 0.691 hit | 0.769 hit | 0.649 hit | 0.769 hit | 0.666 hit |
| Semantic | 0.653 hit | 0.651 hit | 0.654 hit | 0.802 hit | 0.673 hit |
| Sentence window | 0.798 hit | 0.780 hit | 0.656 hit | 0.770 hit | 0.730 miss |

Largest |store_score - cosine_sim| over all 75 rows: 0.000001

### Confidently scored chunks that do NOT contain the answer (from annotations.csv)

| Technique | Question | Rank | Cosine | Source file | Preview |
|---|---|---|---|---|---|
| Sentence window | q1 | 1 | 0.7979 | 'ecfr_21cfr_part7.txt' | (b) On the basis of this determination, the Food and Drug Administration will assign the r |
| Sentence window | q1 | 2 | 0.7830 | 'ecfr_21cfr_part7.txt' | (m) Recall classification means the numerical designation, i.e., I, II, or III, assigned b |
| Sentence window | q5 | 1 | 0.7301 | 'fda_frito-lay-issues-voluntary-allergy-alert.txt' | Those with an allergy or severe sensitivity to milk run the risk of a serious or |
| Sentence window | q5 | 2 | 0.7239 | 'fda_sugar-foods-issues-recall-specific-lots.txt' | Out of an abundance of caution, and because this milk powder was used in a seasoni |
| Semantic | q1 | 1 | 0.6535 | 'fda_fresh-ready-foods-llc-recalls-spicy.txt' | ET. This recall is being conducted in cooperation with the FDA. |

updated /Users/Sunny/Documents/data260-9275/reports/hw03/METRICS.md
```

Technique	Question	Rank	Cosine	Source file	Retrieved text
Sentence window	q5	1	0.7301	fda_frito-lay-issues-voluntary-allergy-alert.txt	Those with an allergy or severe sensitivity to milk run the risk of a serious or life-threatening allergic reaction...
Sentence window	q1	1	0.7979	ecfr_21cfr_part7.txt	(b) On the basis of this determination, the Food and Drug Administration will assign the recall a classification...

Wang — HW3 — 12

2.8 Analysis

Token chunking worked best overall in my test. It found the right source file for all 5 questions (Recall@5 = 1.00), and its Mean@5 cosine was 0.649.

Sentence-window got the highest top-1 cosine (0.747), but its recall was the lowest (0.80). It missed q5 completely. For q5 (the milk with cleaning agents), all 5 results were from other recalls, for example a sentence about people with a milk allergy. For q1, the top 2 results were from the right file (21 CFR Part 7), but they did not have the Class II definition in them. So a high cosine score does not always mean the answer is there. A short sentence with common words like “milk” or “recall” can get a high score even if it does not answer the question.

Semantic chunking also found the right file for all 5 questions (Recall@5 = 1.00), but its top-1 cosine was the lowest (0.687). Its chunk sizes were very uneven. The longest chunk had 11,491 characters, which is much longer than what MiniLM can read (about 256 word pieces), so most of that chunk was cut off when it was embedded. Some other chunks were only one short sentence, like “This recall is being conducted in cooperation with the FDA.” One of these was ranked first for q1 even though it had no useful information.

The best technique was not the same for every question. On q5, semantic chunking ranked the correct document first, and sentence-window failed. On q1 and q2, sentence-window gave the highest scores.

Sentence-window was also the slowest (29.85 ms per search), because it made the most chunks (1,706).

2.9 Conclusion

For this corpus, I think token chunking is the best choice. It had perfect recall, and its chunks were a similar size, so each chunk kept enough context. I also learned that cosine similarity alone is not a good way to judge retrieval. It is better to check whether the retrieved chunks really contain the answer.

Reproducibility and verification

Full steps are in reports/hw03/reproducible_run_instructions.md. In short:

commands

```
conda activate data260 && pip install -r requirements.txt
make run-auth          # Part 1 app on http://localhost:8275
make demo-session      # Part 1 cookie / logout / idle proof
make fetch-corpus       # Part 2 corpus + CORPUS_MANIFEST.json + SOURCES.md
git commit ...         # corpus and questions.yaml BEFORE results
make run-rag           # three pipelines -> raw/, RUN_LOG.txt
make rag-annotations    # annotations.csv to label
make rag-summary        # tables -> METRICS.md
make verify-hw03        # -> verification.json
```

verify_hw03.py checks Part 1 end to end in-process (port, alert, cookie flags, logout replay, idle timeout), the corpus hashes and size, questions.yaml, the shape of raw/, five raw rows chosen with VERIFY_SEED 269275 against their source files, the git commit order, and that the summary recomputes.

```
((data260) Sunny@MacBook-Pro-7 data260-9275 % make verify-hw03
python verify_hw03.py
HW3 verification SID4=9275 PORT_BASE=8275 DOMAIN_ID=3 VERIFY_SEED=269275

/opt/miniconda3/envs/data260/lib/python3.12/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa
[PASS] auth app uses PORT_BASE PORT_BASE=8275
[PASS] home page renders
[PASS] wrong password shows a Bootstrap alert status=401
[PASS] login redirects to /dashboard
[PASS] session cookie is HttpOnly, Secure, SameSite BBKVu4; path=/; Max-Age=3600; httponly; samesite=lax; secure
[PASS] dashboard works while logged in
[PASS] old cookie is rejected after logout status=302 location=/login?reason=ended
[PASS] idle session is rejected status=302 location=/login?reason=ended
[PASS] corpus files match CORPUS_MANIFEST.json hashes 90 files
[PASS] corpus is at least 200 KB 274,233 bytes
[PASS] SOURCES.md exists
[PASS] questions.yaml has 5+ complete questions 5 questions
[PASS] every expected_source is a corpus file
[PASS] 2+ questions depend on a single source document 5 single-source
[PASS] retrievals.jsonl covers 3 techniques x all questions x k 75 rows, expected 75
[PASS] chunk_stats.json has all 3 techniques
[PASS] store_score agrees with recomputed cosine max gap 0.000001
[PASS] random raw rows (VERIFY_SEED) really come from their source file checked 5
[PASS] questions.yaml committed before the retrieval results questions.yaml committed; retrieval results not committed yet
[PASS] summary table recomputes from raw/ 3 techniques
[PASS] METRICS.md contains the generated table

21/21 checks passed -> reports/hw03/verification.json
```

AI_USE

Same answers as reports/hw03/AI_USE.md.

1. What I used an AI assistant for, and what I did myself

I did the design and the checking myself. I decided to use a mixed corpus (FDA recall announcements plus 21 CFR Part 7) instead of only openFDA JSON records, because each record is shorter than one chunk and the three chunkers would give almost the same result. I also decided where the corpus is stored, that the raw output is split into two files, that recall is computed from questions.yaml instead of stored, that sentence-window is measured on the single sentence, that annotations go in a separate file, and that HW2 code stays unchanged. I reviewed and approved the five questions and committed them before the retrieval run. I ran all the code on my own computer, labelled annotations.csv by reading the retrieved chunks, took the screenshots, and wrote the first draft of the analysis and conclusion.

I used Claude to explain the assignment and put the steps in order. It found the FDA and eCFR source pages and collected the list of 89 recall announcement URLs. It wrote the code based on my design decisions: the changes to the instructor's auth starter, the fetch, chunking and summary scripts, verify_hw03.py and the Makefile targets. It drafted questions.yaml for me to review, helped debug problems, made the report template, and made the wording of my analysis simpler.

2. One AI output that was wrong

The first version of `rag_fetch_corpus.py` used Python's built-in `html.parser`. Some FDA pages have broken HTML (a `<div>` inside a `<p>`). Because of this, the parser put the whole announcement inside the first paragraph, and the script saved only the title and the summary box. It also skipped tables.

3. How the problem was found

After the first download, the corpus was 227,584 bytes. I sorted the files by size, and the smallest ones were only about 500 bytes. When I opened `fda_ambriola-company-issues-recall-cheese-products.txt`, it had only the title and summary, but the web page has several paragraphs and two tables of expiration dates.

4. What was changed and why it works now

The script now uses the `lxml` parser, which fixes broken HTML the same way a browser does, and reads every paragraph, list item and table row in page order. After running it again, the corpus is 274,233 bytes, the smallest file is 1.4 KB, and the Ambriola file has the full text and both tables. I also checked that the answer to each of the five questions appears in its expected source file.